

UNIT — 1 : OOP BASICS, DATA TYPES, OPERATORS & CONTROL STATEMENTS

Java Programming — Complete Unit 1 Notes

Course: Object-Oriented Programming through Java | **Year/Sem:** 2nd Year Sem 1 (2-1)

Syllabus Coverage: Program Structure, Simple Java Programs, Tokens, Statements, Command Line Args, Scanner Input, Escape Sequences, Data Types, Type Casting, Variables Scope, Constants, printf(), Static & Final, Operators, and Control Statements.

Source: ChatGPT Curated Exam Preparation Notes

Below are **exam-oriented notes** for all the topics you listed. For each important concept, I'll give **definition + syntax + simple real-time/example program**.

1. Program Structure in Java

Introduction

A Java program generally contains:

1. Class declaration
2. **main()** method
3. Statements
4. Variables
5. Comments

Basic Structure

```
class Demo {  
    public static void main(String[] args) {  
  
        System.out.println("Hello Java");  
  
    }  
}
```

Explanation

- **class Demo** → defines a class.
- **public** → accessible from anywhere.
- **static** → can run without creating an object.
- **void** → does not return a value.
- **main()** → starting point of the program.
- **String[] args** → stores command-line arguments.
- **System.out.println()** → prints output.

2. Writing Simple Java Programs

Example: Student Details

```
class Student {  
    public static void main(String[] args) {  
  
        String name = "Abhiram";  
        int age = 19;  
        double marks = 85.5;  
  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
        System.out.println("Marks: " + marks);  
    }  
}
```

3. Elements / Tokens in Java

Token: The smallest individual unit of a Java program.

Types of Tokens

1. Keywords
2. Identifiers
3. Literals
4. Operators
5. Separators

Example:

```
int age = 20;
```

Token	Example
Keyword	int
Identifier	age
Operator	=
Literal	20
Separator	;

4. Java Statements

A **statement** is an instruction executed by the Java program.

Example

```
int a = 10;  
int b = 20;  
int sum = a + b;  
  
System.out.println(sum);
```

Types include:

- Declaration statement
- Assignment statement
- Expression statement
- Control statement

5. Command Line Arguments

Command-line arguments are values supplied to a program when it is executed.

Syntax

```
public static void main(String[] args)
```

Program

```
class CommandLine {  
    public static void main(String[] args) {  
        System.out.println("Name: " + args[0]);  
        System.out.println("Age: " + args[1]);  
    }  
}
```

Run:

```
java CommandLine Abhiram 19
```

Output:

```
Name: Abhiram  
Age: 19
```

Important: Command-line arguments are stored as **String**.

6. User Input Using Scanner

Scanner is used to read input from the keyboard.

Syntax

```
Scanner sc = new Scanner(System.in);
```

Program

```
import java.util.Scanner;  
  
class StudentInput {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        System.out.print("Enter name: ");  
        String name = sc.nextLine();  
  
        System.out.print("Enter age: ");  
        int age = sc.nextInt();  
  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
    }  
}
```

Common Scanner Methods

Method	Input
<code>nextInt()</code>	integer
<code>nextFloat()</code>	float
<code>nextDouble()</code>	double
<code>next()</code>	one word
<code>nextLine()</code>	complete line
<code>nextBoolean()</code>	true/false

7. Escape Sequences

Escape sequences are special characters beginning with \.

Escape	Meaning
\n	New line
\t	Tab
\"	Double quote
\'	Single quote
\\	Backslash
\b	Backspace

Example

```
class EscapeDemo {  
    public static void main(String[] args) {  
  
        System.out.println("Hello\nJava");  
        System.out.println("Name:\tAbhiram");  
        System.out.println("\"Java Programming\"");  
    }  
}
```

Output:

```
Hello  
Java  
Name:   Abhiram  
"Java Programming"
```

8. Comments

Comments explain the code and are ignored by the compiler.

Single-line

```
// This is a comment  
int age = 20;
```

Multi-line

```
/*  
    This is a  
    multi-line comment  
*/
```

Documentation comment

```
/**  
 * This is documentation.  
 */
```

9. Programming Style

Programming style means writing code in a clear and consistent way.

Good practices

```
class StudentDetails {  
    public static void main(String[] args) {  
        int studentAge = 20;  
        double studentMarks = 85.5;  
  
        System.out.println(studentAge);  
        System.out.println(studentMarks);  
    }  
}
```

Use:

- Meaningful variable names
- Proper indentation
- Comments where necessary
- CamelCase naming
- One statement per line

10. Data Types in Java

A data type specifies what kind of data a variable can store.

Primitive Data Types

Data Type	Example
byte	byte age = 20;
short	short x = 1000;
int	int marks = 90;
long	long population = 100000L;
float	float price = 10.5f;
double	double salary = 25000.50;
char	char grade = 'A';
boolean	boolean pass = true;

11. Declaration of Variables

A variable is a named memory location used to store data.

Syntax

```
dataType variableName;
```

Declaration

```
int age;
```

Initialization

```
age = 20;
```

Declaration + Initialization

```
int age = 20;
```

12. Type Casting

Type casting means converting one data type into another.

A. Widening Casting

Smaller type → larger type.

```
int a = 10;
double b = a;

System.out.println(b);
```

int → **double**

This happens automatically.

B. Narrowing Casting

Larger type → smaller type.

Syntax

```
smallerType variable = (smallerType)value;
```

Example

```
double price = 99.99;

int amount = (int) price;

System.out.println(amount);
```

Output:

```
99
```

The decimal portion is removed.

13. Scope of Variable

Scope means the region where a variable can be accessed.

Local Variable

```
class ScopeDemo {

    public static void main(String[] args) {

        int age = 20;

        System.out.println(age);

    }

}
```

age is available only inside **main()**.

Block Scope

```
if (true) {
    int x = 10;
    System.out.println(x);
}
```

x cannot be accessed outside the **if** block.

14. Literal Constants

A literal is a fixed value directly written in the program.

Examples:

```
int age = 20;
double price = 99.5;
char grade = 'A';
boolean result = true;
String name = "Abhiram";
```

Here:

- **20** → integer literal
- **99.5** → floating-point literal
- **'A'** → character literal
- **true** → boolean literal
- **"Abhiram"** → string literal

15. Symbolic Constants

A symbolic constant is generally declared using **final**.

Syntax

```
final dataType CONSTANT_NAME = value;
```

Example

```
class Circle {
    public static void main(String[] args) {

        final double PI = 3.14159;

        double radius = 5;
        double area = PI * radius * radius;

        System.out.println("Area = " + area);
    }
}
```

PI cannot be changed after initialization.

16. Formatted Output Using printf()

printf() is used to display formatted output.

Syntax

```
System.out.printf("format", value);
```

Example

```
class PrintDemo {
    public static void main(String[] args) {

        String name = "Abhiram";
        int age = 19;
        double marks = 85.5678;

        System.out.printf("Name: %s\n", name);
        System.out.printf("Age: %d\n", age);
        System.out.printf("Marks: %.2f\n", marks);
    }
}
```

Output:

```
Name: Abhiram
Age: 19
Marks: 85.57
```

Important Format Specifiers

Specifier	Meaning
%d	Integer
%f	Floating point
%.2f	2 decimal places
%c	Character
%s	String
%b	Boolean
%n	New line

17. Static Variables and Methods

Static Variable

A static variable belongs to the class rather than individual objects.

Example

```
class Student {  
    static String college = "SRKR Engineering College";  
    String name;  
  
    Student(String name) {  
        this.name = name;  
    }  
  
    void display() {  
        System.out.println(name);  
        System.out.println(college);  
    }  
  
    public static void main(String[] args) {  
        Student s1 = new Student("Abhiram");  
        Student s2 = new Student("Charan");  
  
        s1.display();  
        s2.display();  
    }  
}
```

Both objects share the same **college** variable.

Static Method

```
static void display() {  
    System.out.println("Hello");  
}
```

Calling:

```
display();
```

or:

```
ClassName.display();
```

18. final Attribute

final prevents modification.

Final Variable

```
final int MAX = 100;
```

This is not allowed:

```
MAX = 200;
```

Example

```
class FinalDemo {  
    public static void main(String[] args) {  
  
        final int MAX_MARKS = 100;  
  
        System.out.println(MAX_MARKS);  
    }  
}
```

19. Operators in Java

Operators are symbols used to perform operations on values.

Main types:

1. Assignment
2. Arithmetic
3. Increment/Decrement
4. Relational
5. Logical
6. Bitwise
7. Ternary

20. Assignment Operator

The assignment operator is =.

```
int x = 10;
```

Other assignment operators:

```
+=  
-=  
*=  
/=   
%=
```

Example:

```
int x = 10;  
  
x += 5;  
  
System.out.println(x);
```

Output:

```
15
```

21. Basic Arithmetic Operators

Operator	Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Remainder

Program

```
class Arithmetic {  
    public static void main(String[] args) {  
  
        int a = 20;  
        int b = 6;  
  
        System.out.println(a + b);  
        System.out.println(a - b);  
        System.out.println(a * b);  
        System.out.println(a / b);  
        System.out.println(a % b);  
    }  
}
```

22. Increment and Decrement Operators

++ → increases by 1.

-- → decreases by 1.

Example

```
int x = 10;  
  
x++;  
  
System.out.println(x);
```

Output:

11

Pre-increment

```
++x;
```

First increases, then uses the value.

Post-increment

```
x++;
```

First uses the value, then increases.

Example:

```
int x = 5;  
  
System.out.println(++x);
```

Output:

6

```
int y = 5;

System.out.println(y++);
System.out.println(y);
```

Output:

```
5
6
```

23. Ternary Operator

The ternary operator is a short form of **if-else**.

Syntax

```
condition ? value1 : value2;
```

Example

```
int marks = 75;

String result = (marks >= 40) ? "Pass" : "Fail";

System.out.println(result);
```

Output:

```
Pass
```

24. Relational Operators

Used to compare values.

Operator	Meaning
==	Equal
!=	Not equal
>	Greater
<	Less
>=	Greater/equal
<=	Less/equal

Example:

```
int a = 10;
int b = 20;

System.out.println(a < b);
```

Output:

```
true
```

25. Boolean Logical Operators

Logical operators work with boolean values.

Operator	Meaning		
&&	AND		
,		,	OR

Operator	Meaning		
!	NOT		

Example

```
int age = 20;
boolean id = true;

if (age >= 18 && id) {
    System.out.println("Eligible");
}
```

Both conditions must be true for **&&**.

26. Bitwise Logical Operators

Bitwise operators work at the bit level.

Operator	Meaning	
&	Bitwise AND	
		Bitwise OR
^	Bitwise XOR	
~	Bitwise NOT	

Example:

```
int a = 5;
int b = 3;

System.out.println(a & b);
System.out.println(a | b);
System.out.println(a ^ b);
```

Binary:

```
5 = 101
3 = 011
```

AND:

```
101
011
---
001 = 1
```

So:

```
5 & 3 = 1
```

27. Operator Precedence and Associativity

Precedence determines which operator is performed first.

Example:

```
int result = 10 + 5 * 2;
```

***** has higher precedence than **+**.

Therefore:

```
10 + (5 * 2)
= 20
```

Simple order to remember

```
()  
++ --  
* / %  
+ -  
< > <= >=  
== !=  
&&  
||  
?:  
= += -= ...
```

Associativity

Associativity determines the direction of evaluation when operators have the same precedence.

Example:

```
int x = 10 - 5 + 2;
```

+ and **-** have the same precedence and are evaluated **left to right**:

```
(10 - 5) + 2  
= 7
```

28. if Statement

Used to execute code when a condition is true.

Syntax

```
if (condition) {  
    statements;  
}
```

Example

```
int marks = 80;  
  
if (marks >= 40) {  
    System.out.println("Pass");  
}
```

29. Nested if

An **if** inside another **if**.

Example

```
int marks = 85;  
  
if (marks >= 40) {  
    if (marks >= 75) {  
        System.out.println("Distinction");  
    }  
}
```

30. if-else

Syntax

```
if (condition) {  
    // true  
} else {  
    // false  
}
```

Example

```
int marks = 35;

if (marks >= 40) {
    System.out.println("Pass");
} else {
    System.out.println("Fail");
}
```

31. switch Statement

Used when one value needs to be compared against multiple cases.

Syntax

```
switch (expression) {

    case value1:
        statements;
        break;

    case value2:
        statements;
        break;

    default:
        statements;
}
```

Example: Menu

```
import java.util.Scanner;

class Menu {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("1. Pizza");
        System.out.println("2. Burger");
        System.out.println("3. Biryani");

        System.out.print("Enter choice: ");
        int choice = sc.nextInt();

        switch (choice) {

            case 1:
                System.out.println("Pizza selected");
                break;

            case 2:
                System.out.println("Burger selected");
                break;

            case 3:
                System.out.println("Biryani selected");
                break;

            default:
                System.out.println("Invalid choice");
        }
    }
}
```

32. Iteration Statements

Iteration means **repeating a block of code**.

Java provides:

1. **while**
2. **do-while**
3. **for**
4. enhanced **for** / for-each

33. while Loop

The **while** loop checks the condition **before** executing.

Syntax

```
while (condition) {  
    statements;  
}
```

Example

```
int i = 1;  
  
while (i <= 5) {  
    System.out.println(i);  
    i++;  
}
```

Output:

```
1  
2  
3  
4  
5
```

■genui■{"learning_viz":{"type_id":"LOOP_BREAK_CONTROL_FLOW","initial_values":{"loop_kind":"while","loop_limit":5,"break_value":3}}}

34. do-while Loop

The **do-while** loop executes the body **at least once**, because the condition is checked afterward.

Syntax

```
do {  
    statements;  
} while (condition);
```

Example

```
int i = 1;  
  
do {  
    System.out.println(i);  
    i++;  
} while (i <= 5);
```

35. for Loop

Used when the number of iterations is generally known.

Syntax

```
for (initialization; condition; update) {  
    statements;  
}
```

Example

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

Output:

```
1  
2  
3  
4  
5
```

36. Nested for Loop

A **for** loop inside another **for** loop.

Example: Pattern

```
class Pattern {  
    public static void main(String[] args) {  
  
        for (int i = 1; i <= 5; i++) {  
  
            for (int j = 1; j <= i; j++) {  
                System.out.print("* ");  
            }  
  
            System.out.println();  
        }  
    }  
}
```

Output:

```
*  
* *  
* * *  
* * * *  
* * * * *
```

37. For-Each Loop

Used mainly to access elements of an array or collection one by one.

Syntax

```
for (dataType variable : array) {  
    statements;  
}
```

Example

```
class ForEachDemo {  
    public static void main(String[] args) {  
  
        int[] marks = {75, 80, 90, 85};  
  
        for (int mark : marks) {  
            System.out.println(mark);  
        }  
    }  
}
```

Here:

```
for (int mark : marks)
```

means:

> Take each value from **marks** and store it temporarily in **mark**.

38. break Statement

break immediately terminates the loop or switch.

Example

```
class BreakDemo {
    public static void main(String[] args) {

        for (int i = 1; i <= 10; i++) {

            if (i == 5) {
                break;
            }

            System.out.println(i);
        }
    }
}
```

Output:

```
1
2
3
4
```

When **i == 5**, the loop stops.

39. continue Statement

continue skips the current iteration and moves to the next iteration.

Example

```
class ContinueDemo {
    public static void main(String[] args) {

        for (int i = 1; i <= 5; i++) {

            if (i == 3) {
                continue;
            }

            System.out.println(i);
        }
    }
}
```

Output:

```
1
2
4
5
```

When **i == 3**, only that iteration is skipped.

Quick Revision Table

Topic	Main Point		
class	Defines a class		
main()	Starting point		
Scanner	Takes keyboard input		
Token	Smallest unit of program		
Variable	Stores data		
final	Cannot be changed		
static	Belongs to class		
printf()	Formatted output		
if	Conditional execution		
switch	Multiple choices		
while	Condition checked first		
do-while	Executes at least once		
for	Repetition		
Nested loop	Loop inside loop		
For-each	Traverses array/collection		
break	Stops loop		
continue	Skips current iteration		
&&	Logical AND		
 		 	Logical OR
!	Logical NOT		
&	Bitwise AND		
 	 	Bitwise OR	
^	Bitwise XOR		
?:	Ternary operator		
++	Increment		
--	Decrement		

Most important programs to practice for exams

1. **Scanner** → Student details
2. **Arithmetic operations**
3. **Type casting**
4. **if-else** → Pass/Fail
5. **Nested if** → Grade
6. **switch** → Menu/calculator
7. **for** → 1 to N
8. **Nested for** → Star patterns
9. **while** → Sum of numbers
10. **do-while** → Menu
11. **For-each** → Array elements
12. **break** → Stop at a particular number
13. **continue** → Skip a number

Original ChatGPT Session Reference:

Shared Chat Link: <https://chatgpt.com/share/6ac3dd4e-9e04-83ee-91e4-f9e1d10480ab>

Note: This document compiles the complete B.Tech / Degree 2nd Year 1st Semester (2-1) Java Programming notes for Unit-1, generated directly from the ChatGPT session at the above link.